

Evolutionary Trajectory Fine-Tuning: Reverse-Engineering Algorithmic Progression

Towards an Open-Loop Autonomous R&D Ecosystem and General Innovation Accelerator

Lukas Benda

lukas.benda@boldpivot.cz

April 2026

Abstract

This paper introduces Evolutionary Trajectory Fine-Tuning (ETFT), a novel machine learning paradigm designed to transform Large Language Models (LLMs) from static code generators into autonomous research scientists. Rather than training on disconnected code fragments, ETFT leverages precise, AI-assisted regressions of State-of-the-Art (SOTA) algorithms to teach LLMs the fundamental vectors of architectural innovation. To overcome the limitations of parametric memory (the “knowledge blindspot”), we expand ETFT into an Open-Loop Empirical Architecture. By employing a retrospective Pareto (80/20) Delta Analysis, the system deterministically identifies structural bottlenecks and deploys autonomous research agents to retrieve up-to-date scientific literature and conduct empirical micro-experiments. Failed hypotheses are organically recycled via In-Context Reinforcement Learning, continuously hardening a Probabilistic Heuristic Filter for *in-silico* triage. Ultimately, this self-improving ecosystem lays the foundation for a domain-agnostic General Innovation Accelerator.

1 Introduction

The application of LLMs in software engineering is rapidly evolving, yet models consistently struggle with tasks requiring fundamental architectural innovation under extreme resource constraints—such as the OpenAI Parameter Golf challenge (compressing an LLM into a 16 MB artifact, trained in 10 minutes on an $8\times H100$ cluster).

To synthesize solutions for such problems, LLMs must understand not just how code functions, but how and *why* it evolves. We introduce Evolutionary Trajectory Fine-Tuning (ETFT), a methodology treating algorithms as evolving entities. Furthermore, we address a critical flaw in current generative paradigms: the assumption that an LLM’s pre-trained weights contain the up-to-date scientific knowledge required to push beyond current boundaries. By integrating an Agentic Research Loop and organically generated negative feedback, we propose a comprehensive, closed-loop R&D simulator capable of independent, self-improving discovery.

2 Theoretical Framework: The Discrete Gradient

Let $\mathcal{A}$ represent a set of fully functional, compilable algorithms solving a specific task. We define an objective fitness function $\mathcal{F}(a)$ inversely proportional to validation loss, subject to hard hardware constraints.

An evolutionary trajectory $\mathcal{T}$ is an ordered sequence $(a_0, a_1, \dots, a_n)$, where $\mathcal{F}(a_{i-1}) < \mathcal{F}(a_i)$. The transition $a_{i-1} \rightarrow a_i$ represents a “macro-evolutionary jump”. The foundation of ETFT relies on training the model on these functional regressions, teaching it the trajectory of optimization.

3 Methodology: The Core ETFT Ecosystem

To ensure scalability, the ETFT training corpus is decomposed into specialized datasets generated via an **AI-Assisted Top-Down Regression Pipeline**.

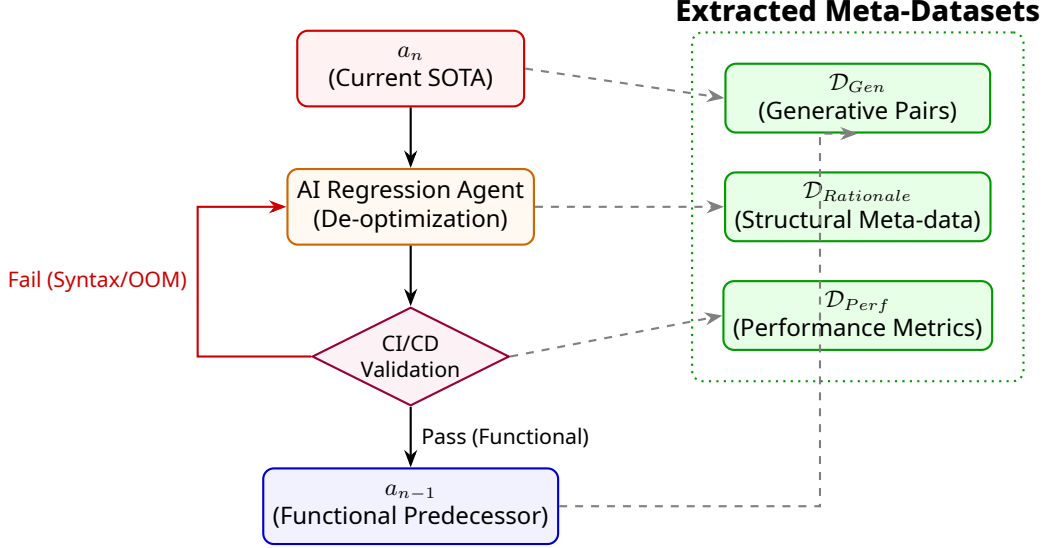


Figure 1: The AI-Assisted Top-Down Regression Pipeline. SOTA algorithms are systematically de-optimized into functional predecessors. Rigorous CI/CD validation ensures code integrity, enabling the automated extraction of the foundational ETFT datasets.

3.1 1. The Generative and Structural Datasets

- **The Generative Trajectory Dataset ($\mathcal{D}_{Gen}$):** Maps the lower-complexity algorithm to its more advanced successor (Input: a_{i-1} , Output: a_i).
- **The Structural-Rationale Meta-Dataset ($\mathcal{D}_{Rationale}$):** Provides a comprehensive textual breakdown of the architectural shift, mapping code perturbations to performance metrics.

3.2 2. The Probabilistic Performance Estimator ($\mathcal{D}_{Perf}$)

Physical evaluation on GPU clusters is computationally expensive. We construct $\mathcal{D}_{Perf}$ to train a **Probabilistic Heuristic Filter**. Acting as an *in-silico triage*, it flags generated SOTA+1 candidates with a high probability of failure (e.g., Out-Of-Memory risks).

Crucially, this filter does not suffer from data asymmetry. It is continuously hardened by the system itself: the probabilistic nature of SOTA+1 hypothesis generation inherently produces high-quality “valuable failures.” These failed variants, safely caught by the triage filter or physical testing, organically populate the negative dataset, dramatically refining the LLM’s predictive accuracy over time.

4 Addressing the Parametric Blindspot: The Open-Loop Architecture

The core assumption that an LLM can extrapolate a SOTA+1 hypothesis relies heavily on its parametric memory, which becomes quickly outdated. To resolve this, we introduce the **Agentic Research Loop (ARL)**, shifting the system to empirical scientific inquiry.

4.1 Retrospective Pareto (80/20) Delta Analysis

Contrary to static, real-time code profiling, the system identifies bottlenecks via a deterministic, retrospective analysis. By analyzing the delta between a_{n-1} and a_n , the system quantifies the character and scope of historical innovations. This provides a rich meta-dataset covering the entire evolutionary history, training the GPT to recognize the sub-components that traditionally yield 80% of the innovation potential. The LLM leverages this learned analytical framework to project the 80/20 focus onto the future $a_n \rightarrow a_{n+1}$ trajectory.

4.2 The Agentic Research Loop (ARL) and RL Feedback

Once the 80/20 bottlenecks are identified, the model formulates targeted **Research Questions**, triggering a two-phase agentic deployment:

1. **Phase 1: Deep Literature Synthesis (Agentic RAG):** Research agents scour the latest preprints and technical reports to retrieve the absolute latest scientific knowledge regarding the specific bottleneck, bypassing the LLM's parametric cut-off.
2. **Phase 2: Empirical Micro-Experiments:** Coding agents execute localized scripts based on retrieved literature to gather physical metrics.

In-Context Reinforcement Learning: Blind alleys encountered during empirical experiments are highly valuable. Failed micro-experiments are immediately recycled as negative reward signals, sharpening the agent's In-Context Reinforcement Learning (RL) to guide and optimize subsequent experimental designs.

5 The Unified Synthesis: Generating SOTA+1

The final generation of the SOTA+1 algorithm is an orchestrated convergence of historical retrospective analysis, real-time literature, and empirically validated RL data.

6 Conclusion: The General Innovation Accelerator

The ETFT architecture represents a paradigm shift in AI-driven software engineering. By integrating the Agentic Research Loop, retrospective 80/20 Delta Analysis, and organic generation of negative data, we overcome the parametric knowledge limits of static models.

The implications of this ecosystem extend far beyond deep learning architectures. An algorithm is fundamentally any set of rules transforming inputs into outputs. Whether mapping the evolution of aerospace alloys in materials science, the progression of targeted therapies in pharmaceuticals, or the optimization of global supply chains, the underlying principle remains the same. By combining historical evolutionary trajectories with autonomous empirical inquiry, probabilistic triage, and continuous In-Context RL feedback, ETFT provides the foundation for a domain-agnostic **General Innovation Accelerator**—teaching machines not just what we know, but the very mechanics of how we invent.

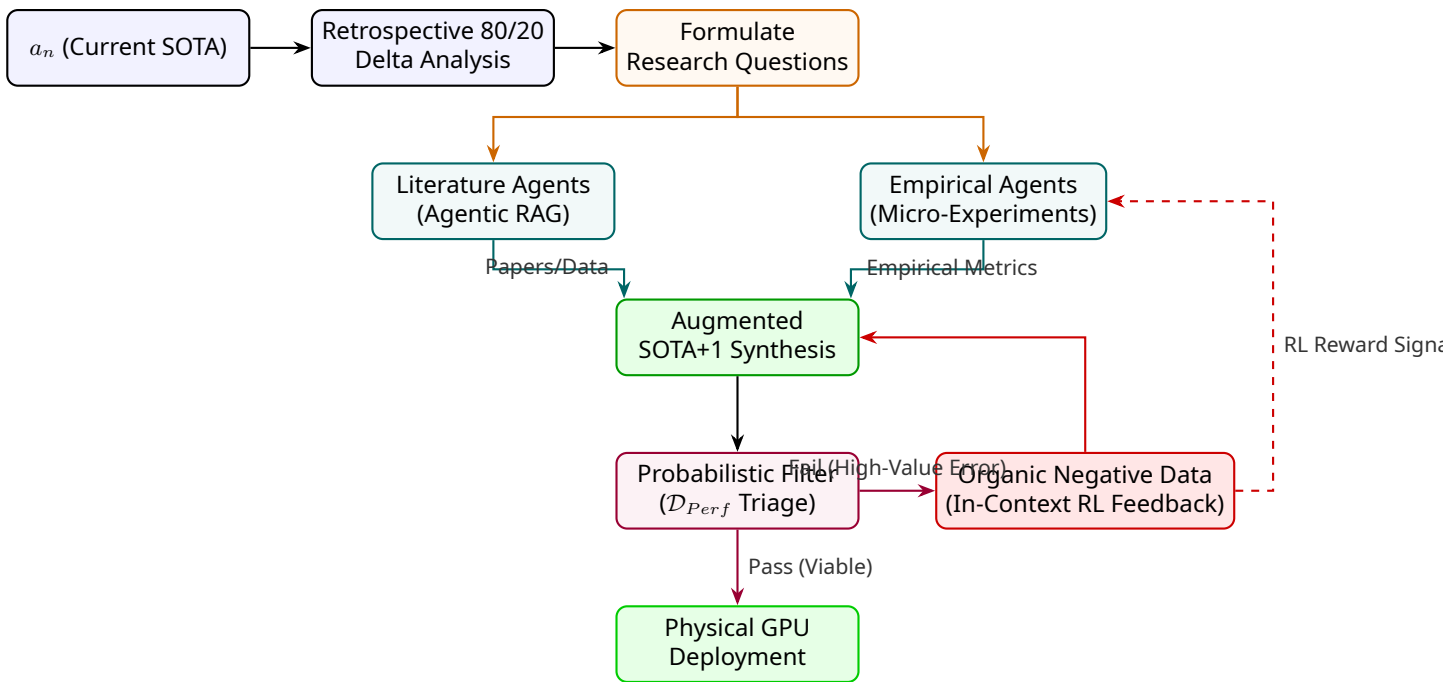


Figure 2: The Open-Loop Empirical ETFT Ecosystem. Failed triage evaluations organically populate negative datasets and route RL reward signals back to the empirical agents, creating a continuously self-improving R&D pipeline.